

CÂY QUYẾT ĐỊNH

Decision Tree — Tiêu chí chia, Overfitting, Cross-Validation & Hyperparameter Tuning

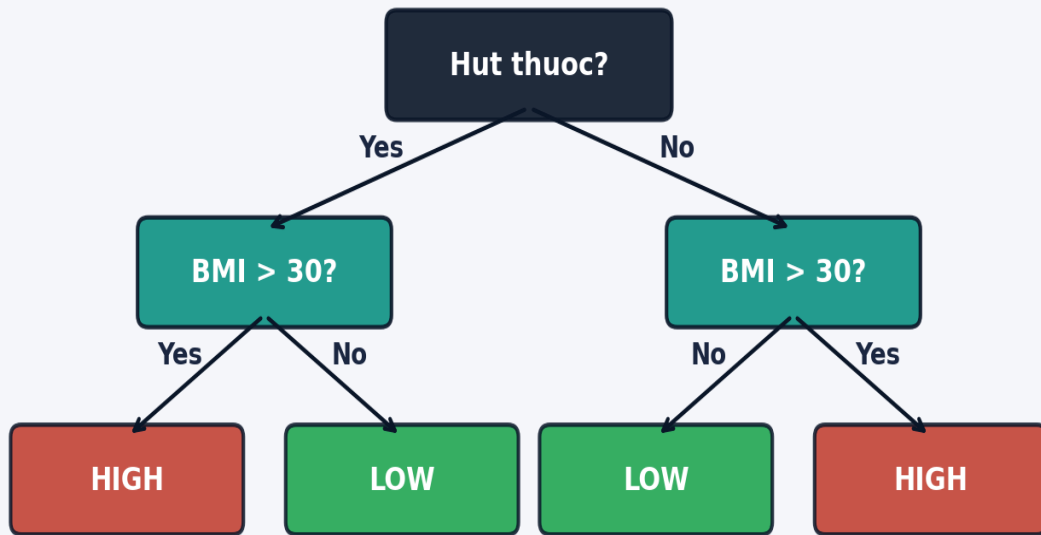
TS. Cao Tiến Dũng



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

Decision Tree là gì?

1



- Mô hình học bằng cách đặt câu hỏi Yes/No liên tiếp về đặc trưng (features)
- Mỗi câu hỏi chia dữ liệu thành 2 nhánh → tinh khiết hơn ở mỗi bước
- Kết quả: tập luật If-Then rõ ràng, con người đọc hiểu được
- Áp dụng cả Hồi quy (regression) và Phân loại (classification)
- **Nền tảng của Random Forest, XGBoost, LightGBM**

Hiểu Decision Tree = hiểu nền tảng của toàn bộ họ Ensemble Methods (Random Forest, Gradient Boosting, XGBoost)

Tại sao học Decision Tree?

2

Đặc điểm	Linear / Logistic Reg.	Decision Tree
Giải thích kết quả	Hệ số β (khó hình dung)	Luật If-Then — Trực quan
Feature Scaling	Bắt buộc	Không cần
Quan hệ phi tuyến	Bị giới hạn	Xử lý tốt
Tương tác feature	Cần tạo thủ công	Tự động học
Overfitting	Ít hơn	Dễ overfit — cần kiểm soát
Nền tảng Ensemble	—	Random Forest, XGBoost

Hiệu DT = hiệu nền tảng Ensemble Methods (Random Forest, Gradient Boosting, XGBoost)



01

Tiêu Chí Chia & Xây Dựng Cây

"Chia để trị — từng bước làm trong sáng hơn"



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Bài toán mẫu — 10 Bệnh nhân

3

ID	Hút thuốc	BMI	Chi phí cao?
1	Yes	32	High
2	Yes	28	High
3	Yes	35	High
4	Yes	25	Low
5	No	29	Low
6	No	27	Low
7	No	24	Low
8	No	22	Low
9	No	29	Low
10	No	33	High

Phân phối gốc (Root)

4 High | 6 Low | Tổng: 10 bệnh nhân

→ Chưa tinh khiết — cần chia

Câu hỏi cần trả lời

Feature nào và ngưỡng nào chia dữ liệu thành 2 nhóm
TINH KHIẾT NHẤT?

2 Feature ứng viên

① Hút thuốc = Yes? ② BMI > 30?

→ Cần thước đo (Gini / Entropy) để so sánh



Độ tinh khiết (Impurity) — Gini & Entropy

4

Gini Impurity

$$\text{Gini}(t) = 1 - \sum_{i=1}^C p_i^2$$

Min = 0.0 (tinh khiết hoàn toàn)

Max = 0.5 (50/50 — hỗn loạn nhất)

Mặc định sklearn: criterion='gini'

Entropy

$$\text{Entropy}(t) = - \sum_{i=1}^C p_i \log_2(p_i)$$

Min = 0.0 bits (tinh khiết hoàn toàn)

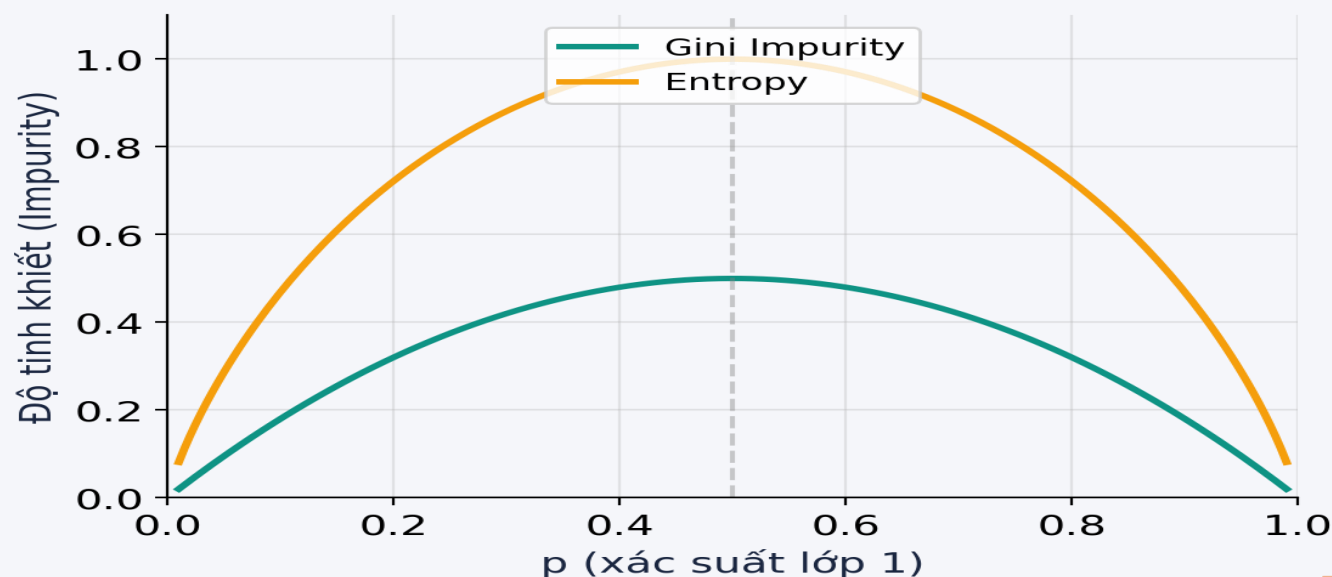
Max = 1.0 bits (50/50 — hỗn loạn nhất)

sklearn: criterion='entropy'

Information Gain

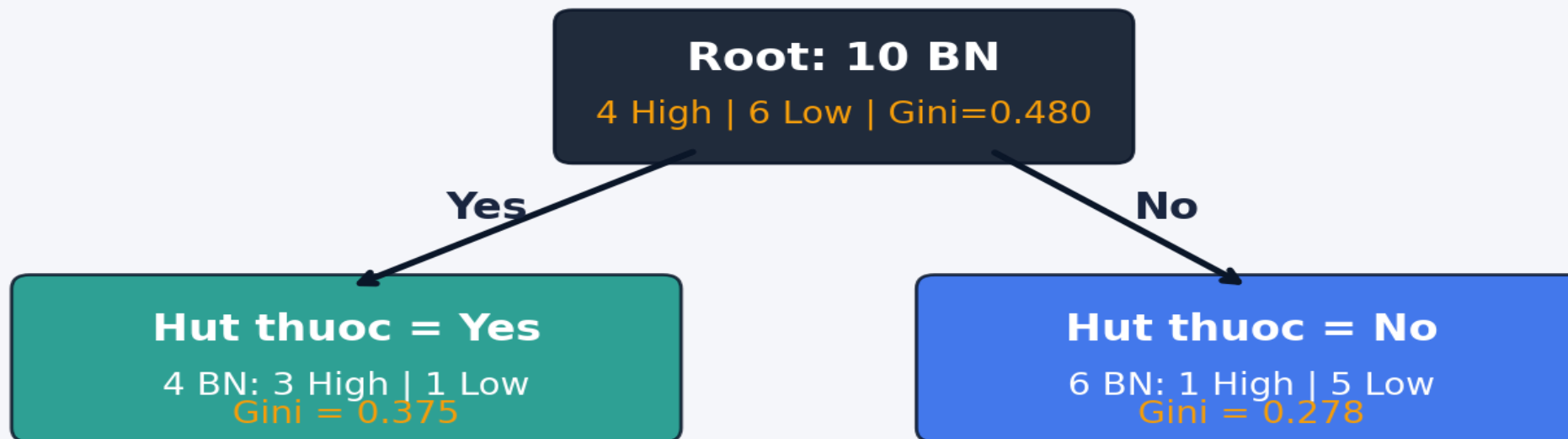
$$\begin{aligned} &= \text{Impurity}(\text{parent}) \\ &\quad - \sum (n_k/n) \times \\ &\quad \text{Impurity}(\text{child}_k) \end{aligned}$$

Chọn feature/ngưỡng có
Gain LỚN NHẤT



Gini Impurity — Tính tay trên 10 bệnh nhân

5



$$\text{Weighted Gini} = 4/10 \times 0.375 + 6/10 \times 0.278 = 0.317$$

$$\text{Gini Gain} = 0.480 - 0.317 = 0.163$$

So sánh: $BMI > 30 \rightarrow \text{Gini Gain} = 0.130 \rightarrow$ Chọn "Hut thuốc" (gain cao hơn)

Gini vs Entropy — Tính tay & So sánh

6

Gini Impurity

Root: $p(H)=0.40$, $p(L)=0.60$

$$\text{Gini} = 1 - (0.40^2 + 0.60^2) = 0.480$$

Nhánh YES (4 người: 3H 1L):

$$\text{Gini} = 1 - (0.75^2 + 0.25^2) = 0.375$$

Nhánh NO (6 người: 1H 5L):

$$\text{Gini} = 1 - (0.167^2 + 0.833^2) = 0.278$$

$$\text{Weighted} = 4/10 \times 0.375 + 6/10 \times 0.278$$

$$= 0.150 + 0.167 = 0.317$$

$$\text{Gini Gain} = 0.480 - 0.317 = 0.163$$

Entropy

Root: $p(H)=0.40$, $p(L)=0.60$

$$\begin{aligned} H &= -0.40 \times \log_2(0.40) - 0.60 \times \log_2(0.60) \\ &= 0.529 + 0.442 = 0.971 \text{ bits} \end{aligned}$$

Nhánh YES (3H 1L):

$$\begin{aligned} H &= -0.75 \times \log_2(0.75) - 0.25 \times \log_2(0.25) \\ &= 0.311 + 0.500 = 0.811 \end{aligned}$$

Nhánh NO (1H 5L):

$$\begin{aligned} H &= -0.167 \times \log_2(0.167) - 0.833 \times \log_2(0.833) \\ &= 0.430 + 0.220 = 0.650 \end{aligned}$$

$$\text{Weighted} = 4/10 \times 0.811 + 6/10 \times 0.650 = 0.714$$

$$\text{Info Gain} = 0.971 - 0.714 = 0.257$$

Cả 2 thước đo đều chọn "Hút thuốc=Yes?" làm split đầu tiên — Gini nhanh hơn (không cần log), Entropy nhạy hơn khi phân phối lệch

Khi feature là số — Cách chọn ngưỡng chia

1

Sắp xếp BMI

[22, 24, 25, 27, 28, 29, 31, 32, 33, 35]

2

Thử tất cả midpoints

Giữa mỗi cặp giá trị liên tiếp:
23, 24.5, 26, 27.5, 28.5, 30, 31.5, 32.5, 34
→ 9 ngưỡng ứng viên (n-1 midpoints)

3

Tính Gini Gain cho mỗi ngưỡng → chọn ngưỡng có Gain lớn nhất

Ngưỡng	Trái ($\leq$)	Phải ($>$)	Gini weighted	Gini Gain
23	1 BN (0H 1L)	9 BN (4H 5L)	0.395	0.085
26	3 BN (0H 3L)	7 BN (4H 3L)	0.350	0.130
28.5	5 BN (1H 4L)	5 BN (3H 2L)	0.400	0.080
30	6 BN (1H 5L)	4 BN (3H 1L)	0.317	0.163
32.5	8 BN (2H 6L)	2 BN (2H 0L)	0.300	0.180
34	9 BN (3H 6L)	1 BN (1H 0L)	0.360	0.120

Best threshold = 32.5 → Gini Gain = 0.180 (cao nhất trong 9 ngưỡng)

Key insight: sklearn tự động thử **TẤT CẢ midpoints** × **TẤT CẢ features** → chọn (feature, threshold) có Gain lớn nhất.

Feature liên tục có NHIỀU ngưỡng hơn feature nhị phân → dễ bị ưu tiên (cardinality bias). *Chi phí:* $O(n \cdot \log n \times p)$ mỗi node.

Bước	Node	Câu hỏi thử	Gini Gain	Quyết định
1	Root (10: 4H 6L Gini=0.480)	Hút thuốc=Yes? vs BMI>30?	0.163 vs 0.130	→ Chọn Hút thuốc=Yes?
2	Nhánh YES (4: 3H 1L, Gini=0.375)	BMI > 30?	0.125	→ Tiếp tục chia
3	Nhánh NO (6: 1H 5L, Gini=0.278)	BMI > 30?	0.111	→ Tiếp tục chia

Thuật toán CART (Classification And Regression Trees)

- 1. Tại mỗi node, duyệt tất cả feature × tất cả ngưỡng → chọn split có Gini Gain lớn nhất
- 2. Chia dữ liệu thành 2 nhánh (trái: điều kiện đúng, phải: điều kiện sai)
- 3. Lặp lại đệ quy trên mỗi nhánh cho đến khi đạt điều kiện dừng
- 4. Điều kiện dừng: node tinh khiết (Gini=0), đạt max_depth, hoặc số mẫu < min_samples_split

02

Kiểm Soát Overfitting

"Cây quá sâu học thuộc lòng — không học được quy luật"



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Overfitting trong Decision Tree

8

Metric	DT (depth=None)	DT (depth=3)	DT (depth=5)	Logistic Reg
Train Accuracy	1.000	0.794	0.819	0.814
Test Accuracy	0.728	0.754	0.765	0.750
CV Accuracy (5-fold)	0.730	0.785	0.776	0.804
Số node lá	287	8	26	—

Overfitting (depth=None)

Train=100% nhưng Test=72.8%
Học thuộc lòng, không tổng quát hoá

Kiểm soát tốt (depth=5)

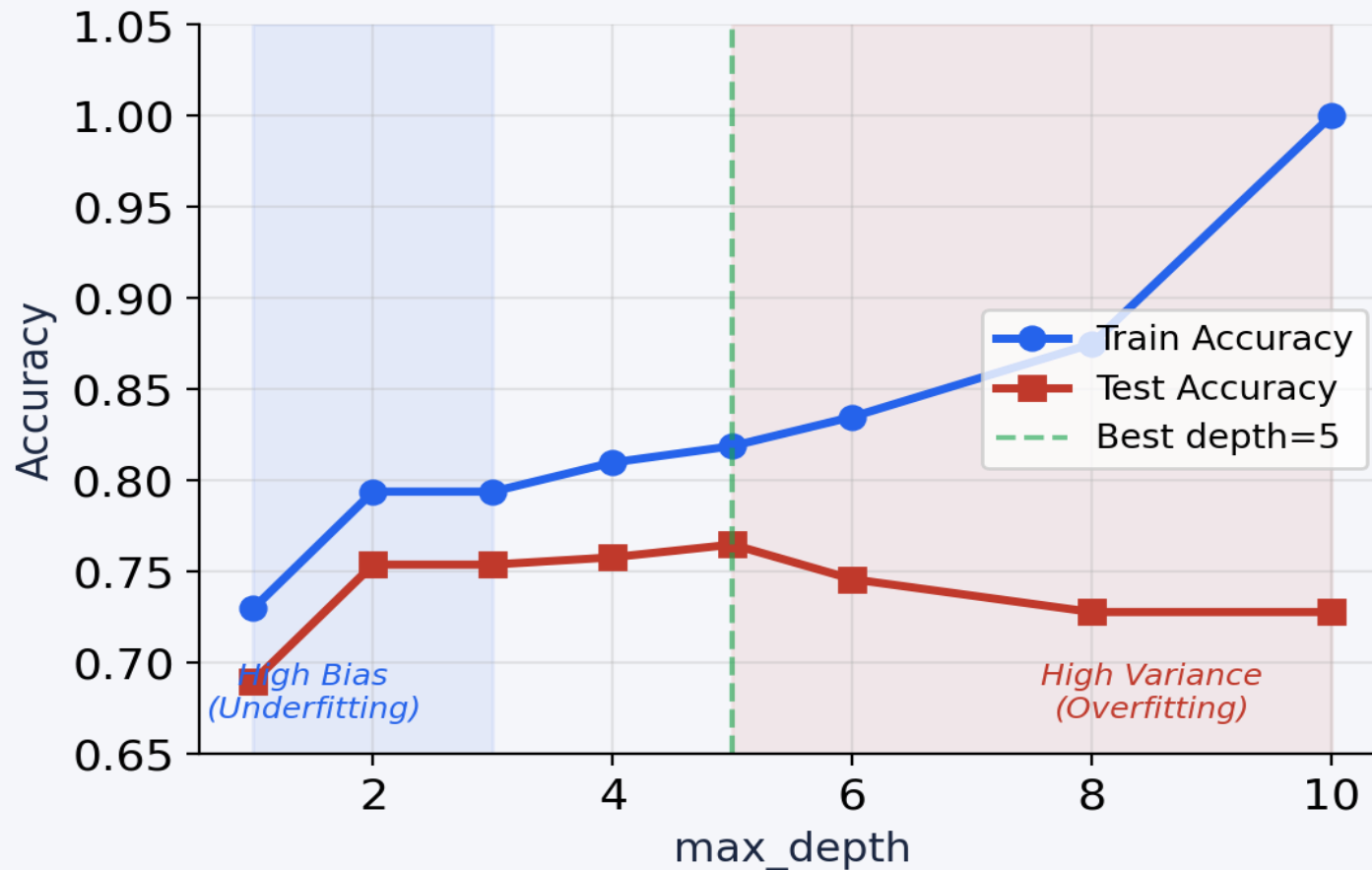
Cân bằng giữa Train và Test
Giữ được khả năng tổng quát hoá

Chiến lược: ① max_depth=3 → ② Tăng dần, vẽ train/test → ③ Dừng khi test giảm → ④ GridSearchCV tối ưu



max_depth — Bias-Variance Tradeoff

9



depth	Train	Test
1	0.730	0.690
2	0.794	0.754
3	0.794	0.754
4	0.810	0.758
5	0.819	0.765
6	0.835	0.746
8	0.875	0.728
None	1.000	0.728

Best depth = 5
Test Acc = 0.765



Tổng hợp Hyperparameter kiểm soát Overfitting

10

Tham số	Ý nghĩa	Tăng →	Giảm →	Khuyến nghị
max_depth	Số tầng tối đa	Underfitting	Overfitting	Thử 3–6
min_samples_leaf	Số mẫu tối thiểu ở lá	Underfitting	Overfitting	Thử 10–50
min_samples_split	Số mẫu tối thiểu để chia	Underfitting	Overfitting	Mặc định=2
max_leaf_nodes	Số lá tối đa	Underfitting	Overfitting	Thay thế max_depth
ccp_alpha	Hệ số cắt tỉa (pruning)	Cây đơn giản	Cây đầy đủ	Tìm qua CV

min_samples_leaf — Kiểm soát kích thước node lá

= 1 (default): Lá chứa 1 mẫu duy nhất → Overfit mạnh — học thuộc lòng

= 20: Mỗi lá ≥ 20 mẫu → Tổng quát tốt hơn, giảm variance

= 50: Mỗi lá ≥ 50 mẫu → Cây đơn giản, ít lá, dùng cho dataset nhỏ

Chiến lược thực hành: ① max_depth=3 → ② Tăng dần, vẽ train/test → ③ Dừng khi test giảm → ④ GridSearchCV



03

Cross-Validation

"Đánh giá mô hình công bằng — không phụ thuộc 1 lần chia"



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

K-Fold Cross-Validation — Ý tưởng & Sơ đồ

11

Test	Train	Train	Train	Train
Train	Test	Train	Train	Train
Train	Train	Test	Train	Train
Train	Train	Train	Test	Train
Train	Train	Train	Train	Test

Fold 1 Acc = 0.785

Fold 2 Acc = 0.776

Fold 3 Acc = 0.790

Fold 4 Acc = 0.782

Fold 5 Acc = 0.788

Mean = 0.784 ± 0.005

Tại sao cần CV?

Train/Test split 1 lần → kết quả phụ thuộc cách chia. K-Fold chia K lần, mỗi fold là Test 1 lần → đánh giá ổn định hơn.

K phổ biến

K=5 hoặc K=10

Report: mean ± std

```
from sklearn.model_selection import cross_val_score  
scores = cross_val_score(DecisionTreeClassifier(max_depth=5), X, y, cv=5, scoring='accuracy')  
print(f'CV Accuracy: {scores.mean():.3f} ± {scores.std():.3f}')
```



K-Fold CV

Chia đều K phần ngẫu nhiên.
Không đảm bảo tỷ lệ lớp.
OK khi data cân bằng.

Stratified K-Fold

Giữ tỷ lệ lớp trong mỗi fold.
Quan trọng khi imbalanced.
Mặc định của sklearn cho
classification.

Repeated K-Fold

Lặp K-Fold nhiều lần với seed khác.
Giảm variance của ước lượng.
Tốn thời gian hơn.

```
from sklearn.model_selection import StratifiedKFold, RepeatedStratifiedKFold, cross_val_score
```

```
# Stratified K-Fold (giữ tỷ lệ lớp)
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
scores = cross_val_score(model, X, y, cv=skf, scoring='accuracy')
```

```
# Repeated Stratified K-Fold (lặp 3 lần × 5-fold = 15 lần đánh giá)
```

```
rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
```

```
scores = cross_val_score(model, X, y, cv=rskf)
```

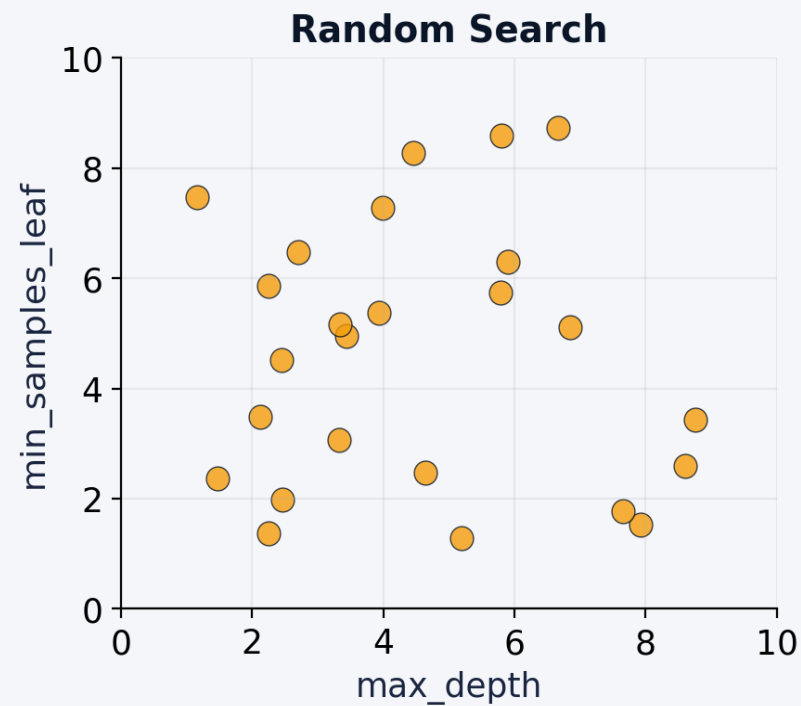
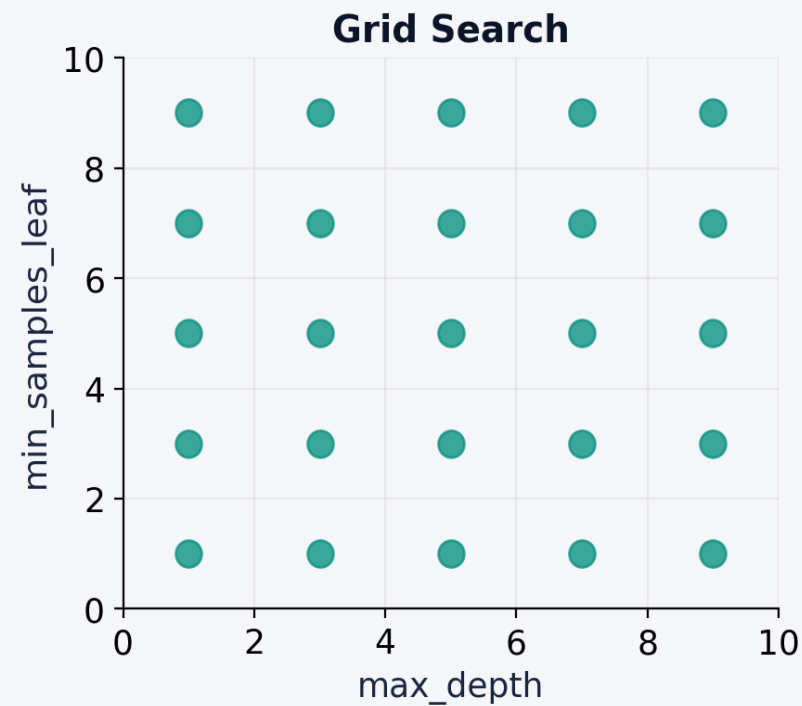
04

Lựa Chọn Mô Hình & Tinh Chỉnh Siêu Tham Số

"Tìm bộ hyperparameter tốt nhất — một cách khoa học"



Grid Search vs Random Search



Ví dụ param_grid

`max_depth: [3, 4, 5, 6]`

`min_samples_leaf: [5, 10, 20, 50]`

`criterion: ['gini', 'entropy']`

→ **Grid: 4×4×2 = 32 tổ hợp**

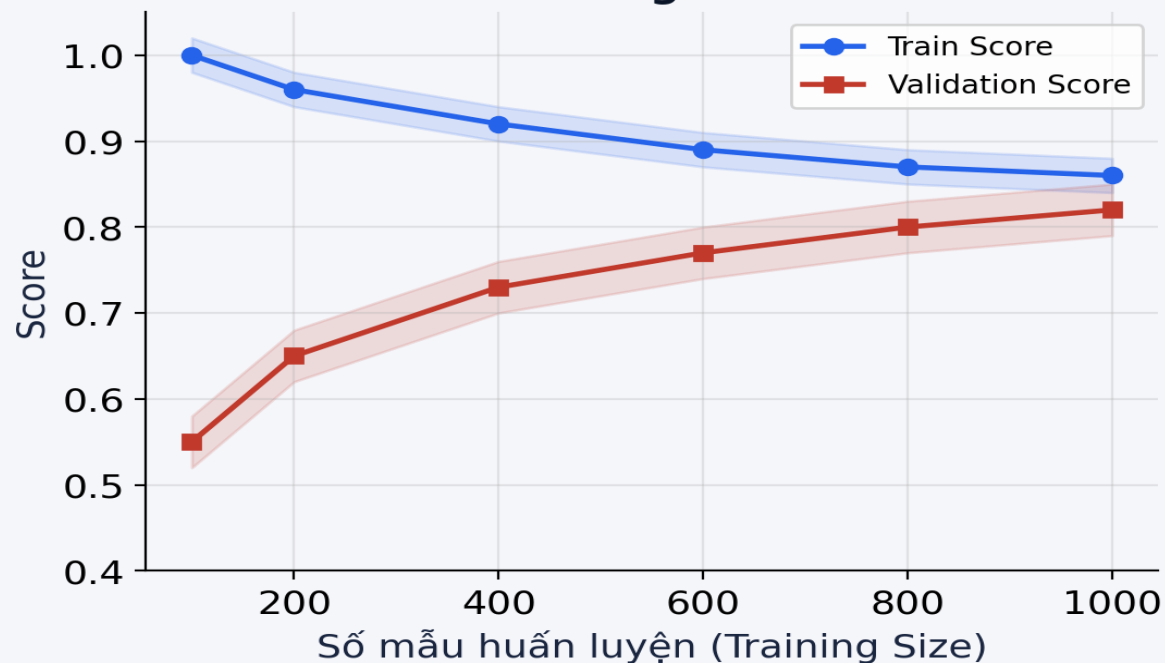
→ **Random (n_iter=20): 20 tổ hợp**

Tiêu chí	Grid Search	Random Search
Cách thức	Thử TẤT CẢ tổ hợp	Chọn NGẪU NHIÊN n tổ hợp
Ưu điểm	Đảm bảo tìm best trong lưới đã định	Nhanh, cover nhiều vùng tham số hơn
Nhược điểm	Chậm khi nhiều tham số (curse of dim.)	Không đảm bảo tìm best tuyệt đối
Khi nào dùng	≤ 2-3 tham số, ít giá trị mỗi tham số	≥ 3 tham số, không gian lớn

Learning Curve & Validation Curve

14

Learning Curve



Learning Curve

Trục X = số mẫu train. Hai đường hội tụ = OK.
Khoảng cách lớn = overfitting. Cả 2 thấp = underfitting.

Validation Curve



Validation Curve

Trục X = giá trị hyperparameter (vd: max_depth).
Chọn điểm validation cao nhất trước khi giảm.



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

05

Thực Hành với Python

"Hands-on: Decision Tree, CV, Curves & Tuning"



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt, numpy as np

# 1. Huấn luyện Decision Tree
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)
print(f'Train Acc: {dt.score(X_train, y_train):.3f}')
print(f'Test Acc: {dt.score(X_test, y_test):.3f}')

# 2. Vẽ Learning Curve
train_sizes, train_scores, val_scores = learning_curve(
    dt, X, y, cv=5, scoring='accuracy',
    train_sizes=np.linspace(0.1, 1.0, 10) # 10% đến 100% dữ liệu
)
plt.plot(train_sizes, train_scores.mean(axis=1), label='Train')
plt.plot(train_sizes, val_scores.mean(axis=1), label='Validation')
plt.fill_between(train_sizes,
    val_scores.mean(1)-val_scores.std(1),
    val_scores.mean(1)+val_scores.std(1), alpha=0.2)
plt.xlabel('Training Size'); plt.ylabel('Accuracy')
plt.legend(); plt.title('Learning Curve — DT depth=5')
```

```
from sklearn.model_selection import validation_curve

# Validation Curve: thay đổi max_depth từ 1 đến 15
param_range = np.arange(1, 16)
train_scores, val_scores = validation_curve(
    DecisionTreeClassifier(random_state=42),
    X, y,
    param_name='max_depth',
    param_range=param_range,
    cv=5,
    scoring='accuracy'
)

# Vẽ biểu đồ
plt.figure(figsize=(8, 5))
plt.plot(param_range, train_scores.mean(axis=1), 'o-', label='Train')
plt.plot(param_range, val_scores.mean(axis=1), 's-', label='Validation')
plt.fill_between(param_range,
    val_scores.mean(1) - val_scores.std(1),
    val_scores.mean(1) + val_scores.std(1), alpha=0.2)
plt.xlabel('max_depth'); plt.ylabel('Accuracy')
plt.title('Validation Curve — max_depth')
best_d = param_range[np.argmax(val_scores.mean(1))]
plt.axvline(best_d, ls='--', color='green', label=f'Best={best_d}')
plt.legend(); plt.show()
```

```
from sklearn.model_selection import (
    cross_val_score, StratifiedKFold, KFold
)
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=5, random_state=42)

# --- K-Fold thường ---
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores_kf = cross_val_score(dt, X, y, cv=kf, scoring='accuracy')
print(f'K-Fold CV:    {scores_kf.mean():.3f} ± {scores_kf.std():.3f}')

# --- Stratified K-Fold (giữ tỷ lệ lớp) ---
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores_skf = cross_val_score(dt, X, y, cv=skf, scoring='accuracy')
print(f'Stratified K-Fold: {scores_skf.mean():.3f} ± {scores_skf.std():.3f}')

# --- So sánh kết quả từng fold ---
for i, (kf_s, skf_s) in enumerate(zip(scores_kf, scores_skf)):
    print(f'  Fold {i+1}: KF={kf_s:.3f}  SKF={skf_s:.3f}')
```



```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import randint

# ===== GridSearchCV =====
param_grid = {
    'max_depth': [3, 4, 5, 6, 7],
    'min_samples_leaf': [5, 10, 20, 50],
    'criterion': ['gini', 'entropy']
} # Tổng: 5×4×2 = 40 tổ hợp × 5-fold = 200 lần fit

grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)
print(f'Best params: {grid.best_params_}')
print(f'Best CV Acc: {grid.best_score_:.3f}')

# ===== RandomizedSearchCV =====
param_dist = {
    'max_depth': randint(2, 15),
    'min_samples_leaf': randint(1, 60),
    'criterion': ['gini', 'entropy']
}

rand = RandomizedSearchCV(DecisionTreeClassifier(random_state=42),
                          param_dist, n_iter=50, cv=5, scoring='accuracy', n_jobs=-1)
rand.fit(X_train, y_train)
print(f'Best params: {rand.best_params_}')
```

01 Ý tưởng Yes/No

Đặt câu hỏi liên tiếp → chia data thành nhóm tinh khiết hơn

02 Gini & Entropy

Tính tay 10 BN: Gini Gain=0.163
chọn "Hút thuốc=Yes?"

03 Kiểm soát Overfitting

max_depth=5 (best),
min_samples_leaf, ccp_alpha,
pruning

04 Cross-Validation

K-Fold, Stratified K-Fold → đánh giá ổn định mean $\pm$ std

05 Lựa chọn mô hình

GridSearchCV,
RandomizedSearchCV → best hyperparameters

06 Learning & Validation Curves

Trực quan hoá bias-variance,
chọn điểm dừng tối ưu